

The Weakness of a Single Decision Tree fragility

A single decision tree, while easy to understand, has a significant weakness: it is **highly sensitive** to small changes in the training data.

- A decision tree is **not robust**.
 - **Example:** Changing just **one** training example out of ten can cause the algorithm to pick a completely different feature for the root node split (e.g., "Whiskers" instead of "Ear Shape").
 - This single change at the root leads to a **completely different tree structure**, which can result in different predictions.
-

The Solution: Tree Ensembles

Instead of relying on one "unstable" tree, a more robust approach is to build **many** different decision trees and have them work together.

- A **tree ensemble** is a collection or "ensemble" of multiple decision trees.
-

How Ensembles Make Predictions: Voting

The core idea of a tree ensemble is to let all the trees vote on the final prediction.

1. A new test example is fed into **every single tree** in the ensemble.
2. Each tree makes its own independent prediction.
3. The final prediction is the class that receives the **majority of the votes**.

Example:

You want to classify a new animal. You run it through your ensemble of 3 trees:

- **Tree 1 predicts:** "Cat"
- **Tree 2 predicts:** "Not Cat"
- **Tree 3 predicts:** "Cat"

The **majority vote** is 2-to-1 for "Cat." Therefore, the ensemble's final prediction is **"Cat"**.

Why Ensembles Work

- **Robustness:** By averaging the "opinions" of many trees, the overall algorithm becomes much **less sensitive** to the individual flaws or quirks of any one tree.
- The final model is more stable and often much more accurate than any single decision tree.

The key challenge, which will be addressed next, is how to build *many different and plausible* trees from the same dataset. This is achieved using a technique called **sampling with replacement**.

Sampling with Replacement 🎁

This is a statistical technique used to create **new, random training sets** from our original dataset. It's the key mechanism that allows us to build many *different* trees for a tree ensemble.

How It Works: The Analogy

Imagine you have a bag containing 4 colored tokens: [Red, Yellow, Green, Blue].

To "sample 4 times with replacement," you would:

1. Reach in, draw one token (e.g., **Green**).
 2. **Put the token back in the bag.** This is the "with replacement" step.
 3. Shake the bag, reach in, and draw a second token (e.g., **Yellow**).
 4. **Put it back.**
 5. Repeat this two more times (e.g., you draw **Blue**, put it back, then draw **Blue** again).
-

The Result

- Your new, sampled set is [Green, Yellow, Blue, Blue].
- Notice two things about this new set:
 1. It contains **duplicates** ("Blue" was picked twice).
 2. It is **missing** some original items ("Red" was never picked).

If you sampled *without* replacement, you would just get the same 4 tokens back in a different order every time. The "with replacement" part is critical for creating a new, statistically different dataset.

Application to Machine Learning

We apply this exact same logic to our training data to create new, slightly different datasets for training each tree in our ensemble.

1. Start with your original training set (e.g., 10 examples).
2. To create **New Training Set 1**, you sample 10 times *with replacement* from the original set.
3. This new set will also have 10 examples, but it will be a random mix of duplicates and will be missing some of the originals.
4. You can repeat this entire process to create **New Training Set 2**, **New Training Set 3**, and so on.

You now have multiple, slightly different training sets, which you can use to build multiple, slightly different decision trees.

The Random Forest Algorithm 🌳🌳🌳

The **Random Forest** is a powerful **tree ensemble** algorithm. It addresses the main weakness of a single decision tree (high sensitivity to small data changes) by building *many* different trees and combining their predictions through a voting process.

This method is significantly more robust and accurate than a single tree.

Step 1: Creating the Ensemble (Bagging)

The first key idea is to create many different trees by training them on slightly different versions of the data. This process is called **bagging** (which stands for **B**ootstrap **A**ggregating).

1. Start with your training set of size M .
2. Set the number of trees to build, B (e.g., $B = 100$).
3. **Loop B times:**
 - a. Create a new, randomized training set (of size M) from the original data using **sampling with replacement**.
 - b. **Train a new decision tree** on this new, sampled dataset.
4. This process results in an "ensemble" of B different decision trees.

Note: Setting B to a larger value (e.g., 64 or 128) generally improves performance, but with diminishing returns. Setting it *too* high (e.g., 1000) will just slow down computation without a meaningful increase in accuracy.

Step 2: The "Random" in Random Forest (Feature Randomization)

The bagging method alone produces "bagged decision trees." A Random Forest adds one more crucial step to **increase the diversity** of the trees.

- **Problem:** Even with different training sets, if one feature is very strong (has the highest information gain), *all* the trees might still choose to split on that same feature at the root. This would make the trees very similar and reduce the benefit of voting.
- **Solution (Feature Randomization):**

When training each tree, at **every node** that is being split:

 1. Do not consider all N available features.
 2. Instead, pick a **random subset of K features** (where $K < N$).
 3. Find the feature with the highest information gain *only from that random subset* and use it to make the split.
- **Rule of Thumb:** A typical choice for K is $K \approx \sqrt{N}$ (the square root of the total number of

features).

This process forces each tree to be fundamentally different, as they are not all allowed to use the same "best" features, making the final ensemble much more powerful.

How a Random Forest Makes Predictions 📦

Once the ensemble is built, making a new prediction is done by **majority vote**:

1. Take the new test example and feed it into all B trees in the forest.
2. Each of the B trees makes its own prediction (e.g., "Cat" or "Not Cat").
3. The final prediction from the Random Forest is the class that receives the **most votes**.

This averaging process makes the final prediction very robust and insensitive to the errors or quirks of any single tree.

Fun Fact:

Q: Where does a machine learning engineer go camping?

A: In a random forest!